

Agentic AI

From single LLM calls to autonomous multi-step reasoning agents

01 What is an AI Agent?

A standard LLM call is one-shot: you send a prompt, get a response. An **AI Agent** uses an LLM as a reasoning engine inside a loop — it can **plan**, **use tools**, **observe results**, and **decide what to do next** — iteratively, until a task is complete.

KEY DIFFERENCE

LLM call: input → output. Agent: input → [think → act → observe → think → act → ...] → final output. Agents can take dozens of steps to complete complex tasks.

02 The ReAct Loop — Core Agent Pattern

ReAct (Reason + Act) is the foundational agent pattern. The LLM alternates between reasoning about the situation and taking actions.

Step	What Happens	Example
Thought	LLM reasons about what to do next	"I need to find the price of AAPL stock first."
Action	LLM calls a tool with specific arguments	search_web(query="AAPL stock price today")
Observation	Tool returns its result	"AAPL is trading at \$189.30"
Thought	LLM reasons about the result	"Good, now I need to compare with MSFT."
Action	Next tool call	search_web(query="MSFT stock price today")
...	...loop continues...	...
Final Answer	LLM produces final response when done	"AAPL (\$189) is lower than MSFT (\$415)"

```
# ReAct Agent with LangChain
from langchain.agents import create_react_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain_community.tools import TavilySearchResults
from langchain import hub

llm = ChatOpenAI(model="gpt-4o-mini")
tools = [TavilySearchResults(max_results=3)]
prompt = hub.pull("hwchase17/react")
```

```

agent = create_react_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

result = agent_executor.invoke(
    {"input": "What is the current price of AAPL and how does it
              compare to 6 months ago?"}
)

```

03 Agent Components

Component	Role	Implementation Examples
LLM (Brain)	Reasons, plans, decides what to do next	GPT-4o, Claude 3.5, Llama 3
Tools	Actions the agent can take in the world	Web search, code exec, DB query, file read/write, API calls
Memory	What the agent remembers	In-context (conversation), Vector DB (semantic recall), External KV store
Planning	How it breaks down complex tasks	ReAct, CoT, Plan-and-Execute, Tree of Thought
Executor	Runs the loop, handles errors	LangChain AgentExecutor, LlamaIndex AgentRunner

04 Agent Memory Types

Memory Type	What It Stores	How It Works	Best For
In-Context (Short-term)	Current conversation	Kept in the prompt/context window	Single session tasks
Episodic (Vector DB)	Past interactions and observations	Embed + store, retrieve by semantic similarity	Long-running agents, recall of past actions
Semantic (Knowledge)	Domain facts and knowledge	RAG over a knowledge base	Research agents, Q&A; agents
Procedural (Tool Use)	How to use tools effectively	Learned via fine-tuning or few-shot examples	Specialized workflow agents

05 Tool Types for Agents

Tool Category	Examples	LangChain Tool / Package
Web Search	Search the internet for current info	TavilySearchResults, DuckDuckGoSearchRun
Code Execution	Run Python/bash code	PythonREPLTool, E2B sandbox
File I/O	Read, write, list files	FileReadTool, FileWriteTool
Database	Query SQL or NoSQL	SQLDatabaseTool, MongoDBTool

APIs	Call external services	Custom tool wrapping requests
Browser	Navigate web pages	PlaywrightBrowserToolkit
Calendar/Email	Schedule, send messages	GmailToolkit, custom SMTP tool
Vector Store	Semantic search over docs	VectorStoreQATool
Human-in-loop	Ask a human for clarification	HumanInputRun

06 Planning Patterns

Plan-and-Execute

First use a "Planner" LLM to generate a step-by-step plan for the full task. Then use an "Executor" LLM to execute each step. Better for complex multi-step tasks where the overall plan is clear upfront.

Tree of Thought (ToT)

Instead of one reasoning path, the agent explores multiple reasoning branches simultaneously, evaluates them, and backtracks when a branch is promising. Much better for math, logic, and planning problems.

Self-Reflection (Reflexion)

After each attempt, the agent generates a verbal reflection on what went wrong and what to try differently. Stores reflections in memory. Significantly improves performance on retry.

07 Agentic AI Tools Ecosystem

Tool	Type	Use It For
LangChain Agents	Framework	ReAct, OpenAI-functions, structured-chat agents. Best ecosystem.
LlamaIndex Agents	Framework	Document-centric agents. Strong for RAG + tool use.
LangGraph	Framework	Graph-based stateful agents with cycles. Multi-agent orchestration.
AutoGen (Microsoft)	Framework	Conversational multi-agent systems. Agents talk to each other.
CrewAI	Framework	Role-based multi-agent crews with tasks and goals. (See Guide 5)
OpenAI Assistants API	API	Built-in threads, tools, file search. Managed agent infrastructure.
Claude Tool Use	API	Anthropic's function-calling for Claude models.
E2B Code Interpreter	Sandbox	Safe code execution environment for agents.
Composio	Tools Hub	150+ pre-built tools for agents (GitHub, Slack, Linear, etc.)
Browserbase	Browser Tool	Hosted browser for web automation agents.